

Lab 2 보고서

Operating System / Lab 2. Data Concurrency

1. 과제 개요

이번 과제의 목표는 Hash Table에 대해 서로 다른 lock 적용 범위를 사용하여 동시성 제어를 구현하고, 그에 따른 correctness와 성능 차이를 비교하는 것이다. 구현 대상은 Without Lock, Coarse-grained Lock, Fine-grained Lock의 세 가지 버전이다.

2. 구현한 소스코드 설명

Hash Table은 bucket 배열과 각 bucket에 연결된 singly linked list로 구성된다. key가 들어오면 hash 함수로 bucket index를 계산하고, 같은 bucket에 여러 key가 들어오면 chaining 방식으로 연결 리스트에 저장한다.

insert 연산은 같은 key가 이미 존재하면 value를 누적하고 upd_cnt를 1 증가시킨다. 같은 key가 없으면 새 node를 생성하여 bucket chain의 head에 삽입한다. lookup은 key가 존재하면 value를 반환하고 없으면 0을 반환한다. remove는 target key를 찾아 unlink한 뒤 node를 삭제한다.

CoarseHashTable은 table 전체를 하나의 mutex로 보호하고, FineHashTable은 bucket마다 별도의 mutex를 두어 접근하는 bucket에 대해서만 lock을 획득한다. traversal은 전체 상태를 일관되게 읽기 위해 모든 bucket lock을 고정된 순서로 획득한 뒤 수행하였다.

3. 문제 풀이

3-1. 멀티 스레드 환경에서 요청 수행 순서를 일관되게 보장하는 방법

공유 데이터에 접근하는 구간을 critical section으로 정의하고 mutex를 통해 상호배제를 보장해야 한다. coarse-grained lock은 table 전체를 하나의 mutex로 직렬화하여 single-thread와 같은 일관성을 쉽게 유지한다. fine-grained lock은 같은 bucket에 대해서만 순서를 보장하며, 서로 다른 bucket에 대한 요청은 동시에 수행될 수 있다.

3-2. 동일 bucket에 대한 Insert와 Remove의 처리 순서 차이

동일 bucket에서 insert와 remove가 동시에 발생하면 lock을 먼저 획득한 thread에 따라 최종 결과가 달라질 수 있다. insert가 먼저 수행되면 이후 remove가 해당 node를 삭제할 수 있고, remove가 먼저 수행되면 아무 일도 하지 못한 뒤 insert가 남을 수 있다. 즉 fine-grained lock은 무결성은 보장하지만 연산의 논리적 선후관계 자체를 고정하지는 않는다.

4. 실험 환경 및 측정 방법

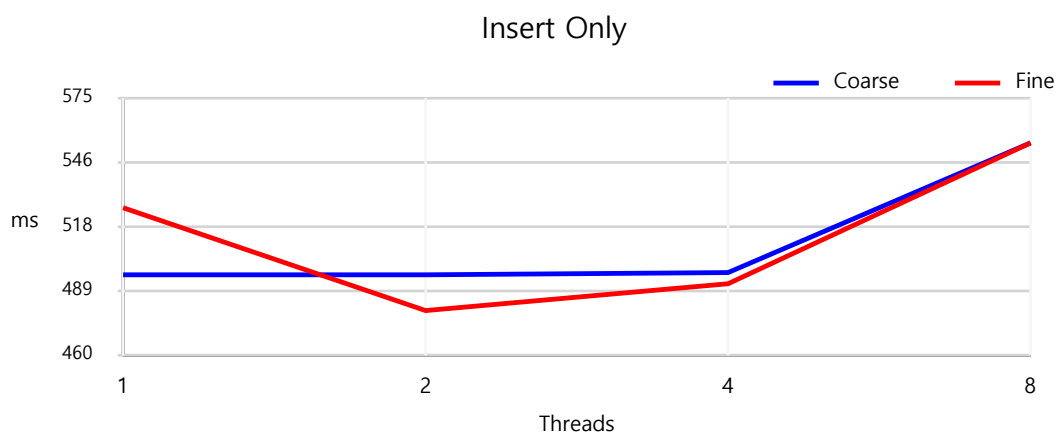
실험은 VirtualBox의 Ubuntu VM에서 수행하였다. 측정은 make clean, make, ./test 순서로 진행했고, 테스트 코드가 출력한 Execution time(ms)를 사용하였다. workload는 Insert Only, Insert + Lookup, Insert + Lookup + Delete의 세 가지이며, thread 수는 1, 2, 4, 8개를 사용하였다.

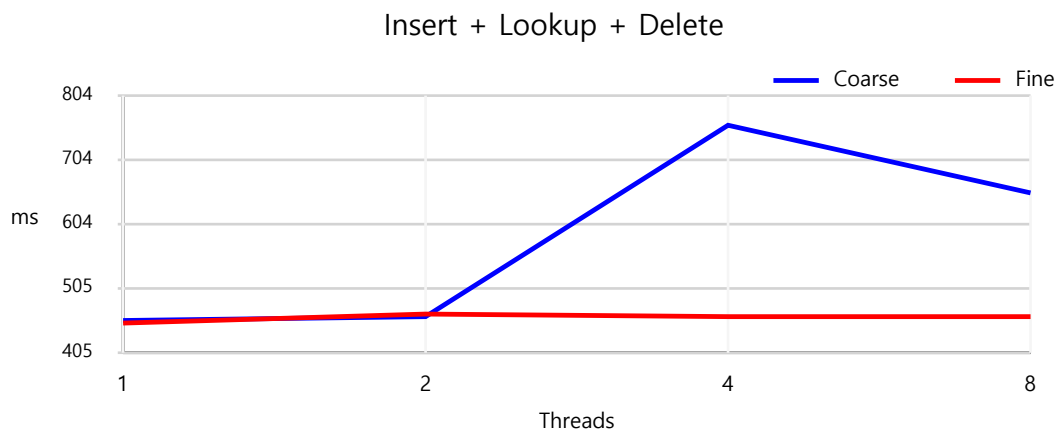
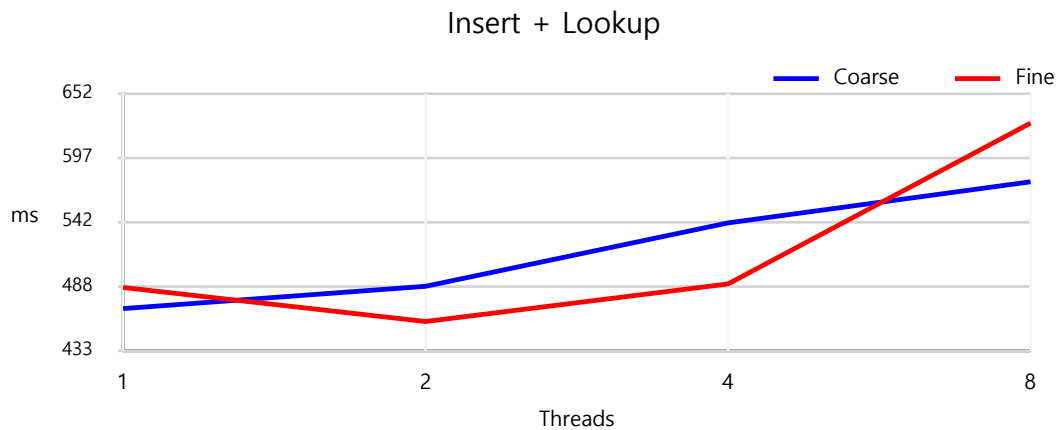
5. 벤치마킹 결과

5-1. 실행 시간 표

Workload	Threads	Coarse (ms)	Fine (ms)
Insert Only	1	496	526
Insert Only	2	496	480
Insert Only	4	497	492
Insert Only	8	555	555
Insert + Lookup	1	469	487
Insert + Lookup	2	488	458
Insert + Lookup	4	542	490
Insert + Lookup	8	577	627
Insert + Lookup + Delete	1	455	451
Insert + Lookup + Delete	2	461	465
Insert + Lookup + Delete	4	758	461
Insert + Lookup + Delete	8	653	461

5-2. 실행 시간 그래프





6. Discussion

6-1. Thread 수에 따른 실행 시간 비교

single-thread 환경에서는 coarse-grained와 fine-grained의 차이가 크지 않았고, 오히려 coarse가 약간 더 빠른 경우도 있었다. 이는 fine-grained 방식이 bucket별 lock 배열을 관리하는 추가 비용을 가지기 때문이다. 반면 multi-thread 환경에서는 workload 종류에 따라 fine-grained lock의 장점이 뚜렷하게 나타났다.

6-2. Fine-grained가 확실히 유리했던 workload

이번 측정에서 fine-grained가 가장 확실하게 유리했던 경우는 Insert + Lookup + Delete workload였다. 4-thread에서는 coarse 758ms, fine 461ms였고, 8-thread에서는 coarse 653ms, fine 461ms였다. 수정 연산이 많고 경쟁이 심한 workload에서는 fine-grained가 coarse-grained보다 확실히 유리할 것이라는 가설을 세울 수 있었고, 이번 결과는 이 가설을 강하게 지지했다.

6-3. Fine-grained가 오히려 느렸던 경우와 가설

Insert + Lookup workload의 8-thread에서는 coarse 577ms, fine 627ms로 fine-grained가 오히려 느렸다. 이에 대한 가설은 key가 일부 bucket에 집중되었거나, lookup이 섞이면서 lock 획득과 해제 빈도가 크게 증가해 bucket별 lock 관리 비용이 더 커졌다는 것이다. 이 가설은 부분적으로 맞았다고 볼 수 있다. 이유는 같은 workload의 2-thread와 4-thread에서는 fine-grained가 더 빨랐기 때문이다. 따라서 fine-grained는 평균적으로 병렬성에 유리하지만, 항상 무조건 빠르지는 않다는 해석이 이번 결과와 가장 잘 맞는다.

6-4. 그래프를 통해 해석할 포인트

그래프를 보면 Insert + Lookup + Delete에서는 thread 수가 증가할수록 coarse-grained의 실행 시간이 크게 증가하는 반면, fine-grained는 비교적 안정적으로 유지된다. 반대로 Insert + Lookup에서는 8-thread에서 fine-grained 선이 coarse-grained보다 위로 올라가는데, 이 점을 근거로 lock granularity를 줄이는 것만으로 항상 성능이 좋아지는 것은 아니라는 점을 설명할 수 있다.

7. 새롭게 배운 점 / 어려웠던 점

이번 과제를 통해 같은 자료구조라도 lock을 어디에 적용하느냐에 따라 correctness와 성능이 크게 달라진다는 점을 배울 수 있었다. 특히 fine-grained lock 구현에서는 동시성 제어와 자료구조 조작을 함께 생각해야 했고, remove와 traversal처럼 포인터 수정 및 전체 일관성이 필요한 연산에서 더 많은 주의가 필요했다.

8. 결론

이번 과제에서는 Hash Table을 기반으로 lock이 없는 버전, coarse-grained lock 버전, fine-grained lock 버전을 구현하고 비교하였다. 실험 결과를 종합하면 fine-grained lock은 대부분의 multi-thread workload에서 더 좋은 확장성을 보였고, 특히 delete가 포함된 경쟁적인 workload에서 강점을 보였다. 반면 workload 특성과 key 분포에 따라 coarse-grained보다 느릴 수도 있으므로, 동시성 자료구조 설계에서는 correctness뿐 아니라 lock 범위와 병렬성의 균형도 함께 고려해야 한다.